

Experiment

‘p_6_1_b_process_array_every_even_element_array’ Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 3

Hard fails (resolution): 1

Soft fails (resolution): 0

Verification attempts: 2

Problem Specification

Problem name: p_6_1_b_process_array_every_even_element_array

Natural language statement: Write a method that takes an array of ten integers and then returns every even element.

Method signature: p_6_1_b_process_array_every_even_element_array(inputs: array<int>) returns (even_elements: array<int>)

Ensures

- even_elements[..] == evens(inputs[..])

Requires

- inputs.Length == 10

Functional Code Given

```
function evens(inputs: seq<int>): seq<int>
{
  if inputs == [] then []
  else if inputs[0] % 2 == 0 then [inputs[0]] + evens(inputs[1..])
  else evens(inputs[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes an array of ten integers and then returns every even element.

The signature should be:

```
p_6_1_b_process_array_every_even_element_array(inputs: array<int>) returns (even_elements  
    ↪ : array<int>)
```

The method should respect the following contract:

```
requires inputs.Length == 10, ensures even_elements[..] == evens(inputs[..])
```

The contract uses the following dafny code:

```
function evens(inputs: seq<int>): seq<int>  
{  
  if inputs == [] then []  
  else if inputs[0] % 2 == 0 then [inputs[0]] + evens(inputs[1..])  
  else evens(inputs[1..])  
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
 ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
 ↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn0p0pBMtqR5oakRZNrpLLjuffyNb

Model: gpt-5.5-2026-04-23

Created at: 1780573602

Finish reason: stop

Usage: 458 tokens in, and 7010 tokens out

```
{  
  var s0 := inputs[0..10];  
  var s1 := inputs[1..10];  
  var s2 := inputs[2..10];  
  var s3 := inputs[3..10];  
  var s4 := inputs[4..10];  
  var s5 := inputs[5..10];  
  var s6 := inputs[6..10];  
  var s7 := inputs[7..10];
```

```

var s8 := inputs[8..10];
var s9 := inputs[9..10];
var s10 := inputs[10..10];

assert inputs[..] == s0;
assert |s0| == 10;
assert |s1| == 9;
assert |s2| == 8;
assert |s3| == 7;
assert |s4| == 6;
assert |s5| == 5;
assert |s6| == 4;
assert |s7| == 3;
assert |s8| == 2;
assert |s9| == 1;
assert |s10| == 0;
assert s0[1..] == s1;
assert s1[1..] == s2;
assert s2[1..] == s3;
assert s3[1..] == s4;
assert s4[1..] == s5;
assert s5[1..] == s6;
assert s6[1..] == s7;
assert s7[1..] == s8;
assert s8[1..] == s9;
assert s9[1..] == s10;
assert s10 == [];

var r10: seq<int> := [];
var r9: seq<int> := if s9[0] % 2 == 0 then [s9[0]] + r10 else r10;
var r8: seq<int> := if s8[0] % 2 == 0 then [s8[0]] + r9 else r9;
var r7: seq<int> := if s7[0] % 2 == 0 then [s7[0]] + r8 else r8;
var r6: seq<int> := if s6[0] % 2 == 0 then [s6[0]] + r7 else r7;
var r5: seq<int> := if s5[0] % 2 == 0 then [s5[0]] + r6 else r6;
var r4: seq<int> := if s4[0] % 2 == 0 then [s4[0]] + r5 else r5;
var r3: seq<int> := if s3[0] % 2 == 0 then [s3[0]] + r4 else r4;
var r2: seq<int> := if s2[0] % 2 == 0 then [s2[0]] + r3 else r3;
var r1: seq<int> := if s1[0] % 2 == 0 then [s1[0]] + r2 else r2;
var r0: seq<int> := if s0[0] % 2 == 0 then [s0[0]] + r1 else r1;

even_elements := new int[|r0|](i => r0[i]);
assert even_elements[..] == r0;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_6_1_b_process_array_every_even_element_array/program.dfy(11,0): Error: a
  ↪ postcondition could not be proved on this return path
  |

```

```

11 | {
    | ^

problem_data/p_6_1_b_process_array_every_even_element_array/program.dfy(10,27): Related
    ↪ location: this is the postcondition that could not be proved
    |
10 | ensures even_elements[..] == evens(inputs[..])
    | ^^

problem_data/p_6_1_b_process_array_every_even_element_array/program.dfy(60,40): Error:
    ↪ index out of range
    |
60 | even_elements := new int[|r0|](i => r0[i]);
    | ^

```

Dafny program verifier finished with 2 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn0s2qtdvkC8trlXGKjGfDWTimpE3

Model: gpt-5.5-2026-04-23

Created at: 1780573790

Finish reason: stop

Usage: 1572 tokens in, and 8639 tokens out

```

{
  var r: seq<int> := [];
  var k := inputs.Length;
  assert k == 10;
  assert inputs[k..] == [];
  assert r == evens(inputs[k..]);

  while k > 0
    invariant inputs.Length == 10
    invariant 0 <= k <= inputs.Length
    invariant r == evens(inputs[k..])
  {
    k := k - 1;
    assert 0 <= k < inputs.Length;
    assert 0 <= k + 1 <= inputs.Length;

    var tail := r;
    assert tail == evens(inputs[k + 1..]);

    var current := inputs[k..];

```

```

    assert current != [];
    assert current[0] == inputs[k];
    assert current[1..] == inputs[k + 1..];

    if inputs[k] % 2 == 0 {
        assert current[0] % 2 == 0;
        assert evens(current) == [current[0]] + evens(current[1..]);
        r := [inputs[k]] + tail;
        assert [current[0]] == [inputs[k]];
        assert evens(current[1..]) == tail;
        assert r == evens(current);
    } else {
        assert current[0] % 2 != 0;
        assert evens(current) == evens(current[1..]);
        r := tail;
        assert evens(current[1..]) == tail;
        assert r == evens(current);
    }
    assert current == inputs[k..];
    assert r == evens(inputs[k..]);
}

assert k == 0;
assert inputs[k..] == inputs[..];
assert r == evens(inputs[..]);

even_elements := new int[|r|](i => if 0 <= i < |r| then r[i] else 0);
assert even_elements.Length == |r|;
assert forall j | 0 <= j < |r|
    ensures even_elements[..][j] == r[j]
{
    assert even_elements[j] == r[j];
    assert even_elements[..][j] == even_elements[j];
}
assert even_elements[..] == r;
assert even_elements[..] == evens(inputs[..]);
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_6_1_b_process_array_every_even_element_array/program.dfy(67,0): Error:

```

    ↪ invalid UnaryExpression
    |
67 | }
    | ^

```

problem_data/p_6_1_b_process_array_every_even_element_array/program.dfy(66,47): Error:

```

    ↪ expected either 'by' or a semicolon following the assert expression
    |

```

```
66 | assert even_elements[..] == evens(inputs[..]);  
    | ^
```

2 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the

- ↪ Dafny body, including the curly braces that surround it. Do not show the signature
- ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn0v24jVYmtLka8mJ7W9qnmVrKY8z

Model: gpt-5.5-2026-04-23

Created at: 1780573976

Finish reason: stop

Usage: 2239 tokens in, and 2755 tokens out

```
{  
  var r: seq<int> := [];  
  var k := inputs.Length;  
  assert k == 10;  
  assert inputs[k..] == [];  
  assert r == evens(inputs[k..]);  
  
  while k > 0  
    invariant inputs.Length == 10  
    invariant 0 <= k <= inputs.Length  
    invariant r == evens(inputs[k..])  
  {  
    k := k - 1;  
    assert 0 <= k < inputs.Length;  
    assert 0 <= k + 1 <= inputs.Length;  
  
    var tail := r;  
    assert tail == evens(inputs[k + 1..]);  
  
    var current := inputs[k..];  
    assert current != [];  
    assert current[0] == inputs[k];  
    assert current[1..] == inputs[k + 1..];  
  
    if inputs[k] % 2 == 0 {  
      assert current[0] % 2 == 0;  
      assert evens(current) == [current[0]] + evens(current[1..]);  
      r := [inputs[k]] + tail;  
      assert [current[0]] == [inputs[k]];  
      assert evens(current[1..]) == tail;  
      assert r == evens(current);  
    } else {  
      assert current[0] % 2 != 0;
```

```

    assert evens(current) == evens(current[1..]);
    r := tail;
    assert evens(current[1..]) == tail;
    assert r == evens(current);
  }
  assert current == inputs[k..];
  assert r == evens(inputs[k..]);
}

assert k == 0;
assert inputs[k..] == inputs[..];
assert r == evens(inputs[..]);

even_elements := new int[|r|](i => if 0 <= i < |r| then r[i] else 0);
assert even_elements.Length == |r|;

forall j | 0 <= j < |r|
  ensures even_elements[..][j] == r[j]
{
  assert even_elements[j] == r[j];
  assert even_elements[..][j] == even_elements[j];
}

assert |even_elements[..]| == |r|;
assert even_elements[..] == r;
assert even_elements[..] == evens(inputs[..]);
}

```

Final Program

```

function evens(inputs: seq<int>): seq<int>
{
  if inputs == [] then []
  else if inputs[0] % 2 == 0 then [inputs[0]] + evens(inputs[1..])
  else evens(inputs[1..])
}

method p_6_1_b_process_array_every_even_element_array(inputs: array<int>) returns (
  ↪ even_elements: array<int>)
  requires inputs.Length == 10
  ensures even_elements[..] == evens(inputs[..])
{
  var r: seq<int> := [];
  var k := inputs.Length;
  assert k == 10;
  assert inputs[k..] == [];
  assert r == evens(inputs[k..]);

  while k > 0
    invariant inputs.Length == 10

```

```

invariant 0 <= k <= inputs.Length
invariant r == evens(inputs[k..])
{
  k := k - 1;
  assert 0 <= k < inputs.Length;
  assert 0 <= k + 1 <= inputs.Length;

  var tail := r;
  assert tail == evens(inputs[k + 1..]);

  var current := inputs[k..];
  assert current != [];
  assert current[0] == inputs[k];
  assert current[1..] == inputs[k + 1..];

  if inputs[k] % 2 == 0 {
    assert current[0] % 2 == 0;
    assert evens(current) == [current[0]] + evens(current[1..]);
    r := [inputs[k]] + tail;
    assert [current[0]] == [inputs[k]];
    assert evens(current[1..]) == tail;
    assert r == evens(current);
  } else {
    assert current[0] % 2 != 0;
    assert evens(current) == evens(current[1..]);
    r := tail;
    assert evens(current[1..]) == tail;
    assert r == evens(current);
  }
  assert current == inputs[k..];
  assert r == evens(inputs[k..]);
}

assert k == 0;
assert inputs[k..] == inputs[..];
assert r == evens(inputs[..]);

even_elements := new int[|r|](i => if 0 <= i < |r| then r[i] else 0);
assert even_elements.Length == |r|;

forall j | 0 <= j < |r|
  ensures even_elements[..][j] == r[j]
{
  assert even_elements[j] == r[j];
  assert even_elements[..][j] == even_elements[j];
}

assert |even_elements[..]| == |r|;
assert even_elements[..] == r;
assert even_elements[..] == evens(inputs[..]);
}

```


Total Token Usage

Input tokens: 4269

Output tokens: 18404

Reasoning tokens: 16248

Sum of ‘total tokens’: 22673

Experiment Timings

Overall Experiment started at 1780573601760, ended at 1780574054779, lasting 453019ms (453.02 seconds)

Iteration #1 started at 1780573601761, ended at 1780573789121, lasting 187360ms (187.36 seconds)

Iteration #2 started at 1780573789121, ended at 1780573975402, lasting 186281ms (186.28 seconds)

Iteration #3 started at 1780573975490, ended at 1780574054778, lasting 79288ms (79.29 seconds)